

西南石油大学本科毕业论文（初稿）

图书馆自习座位智能预约系统

Library Self-study Seat Intelligent Reservation System

题目	图书馆自习座位智能预约系统
学生姓名	陈信豪
学号	202231061133
学院	计算机与软件学院
专业年级	物联网工程 2022 级
指导教师	梁宗文
论文性质	本科毕业论文初稿
完成日期	2026 年 5 月

本文依据当前仓库中的真实工程实现整理撰写，重点围绕微信登录、座位查询、预约、签到、设备感知、实时同步与信誉约束等已具备代码基础的能力展开。

中文摘要

随着高校学习空间数字化建设持续推进，图书馆座位资源管理正从“人工巡查、现场占座”的粗放方式逐步转向“在线预约、实时感知、规则约束”的精细化方式。围绕学生到馆前难以及时获知空闲座位、预约后到场约束不足、座位展示状态与现场情况不同步等实际问题，本文结合当前仓库中的真实项目实现，对图书馆自习座位智能预约系统论文初稿进行了二次增强整理。系统以微信小程序为主要用户入口，以 NestJS 后端为业务中枢，并通过 MQTT 设备上报与 WebSocket 实时推送形成基础业务闭环，目标是在不夸大功能边界的前提下，为图书馆座位管理提供一个具有继续完善价值的工程原型。

项目整体采用“前端应用层—后端服务层—设备感知层”的分层结构。后端基于 NestJS 构建模块化服务，联合 TypeORM、MySQL 与 Redis 完成业务建模、数据持久化与并发控制；预约模块负责预约创建、取消、超时释放与历史查询，签到模块负责二维码与位置校验，座位模块负责状态更新、区域查询与状态日志记录，统计模块提供按日期区间查询、区域热力查询以及日统计生成能力；实时通信部分以 WebSocket 推送 `seat:status`、`reservation:expired` 与 `checkin:reminder` 等事件；设备侧则基于 ESP32 与 HC-SR501 红外传感器采集占用变化，通过 MQTT 将座位感知状态上报给服务器。小程序端采用 Taro 与 React 实现座位查看、预约、签到、历史记录、信誉分入口等页面，并通过 Socket 连接接收后端状态广播。

从当前代码可验证的实现情况来看，系统已经完成 30 分钟预约保留、预约过期释放、基础签到校验、信誉分扣减、座位状态日志记录、设备状态上报和小程序端基础实时同步等核心链路。与此同时，离座超过 1 小时自动释放、完整收藏功能、成熟个人画像统计、完善运营分析、稳定的签到提醒触达机制以及后台管理端等内容仍处于待完善或预留阶段，本文均采取保守描述，不将其写成已完整落地能力。

本文的写作重点并非凭空扩写业务设想，而是以仓库中已有模块、服务、页面、实体与测试文件为依据，形成更接近毕业论文初稿形态的结构化文档。论文围绕系统背景、相关技术、系统设计、系统实现、系统测试与总结展望六个方面展开，希望为后续正式成稿、补图补表、补充文献与进一步联调提供可继续润色的基础文本。

关键词：图书馆；座位预约；微信小程序；NestJS；MQTT；WebSocket

Abstract

With the continuous digital transformation of campus learning spaces, library seat management is gradually shifting from manual inspection and informal occupation to online reservation, real-time sensing, and rule-based coordination. To address practical issues such as poor visibility of available seats, weak arrival constraints after reservation, and inconsistency between displayed seat status and actual on-site conditions, this thesis rewrites the project draft around the real implementation in the current repository and presents a more complete preliminary thesis for an intelligent self-study seat reservation system.

The system adopts a layered structure composed of a mini-program client, backend service modules, and sensing devices. The backend is built with NestJS and integrates TypeORM, MySQL, and Redis for data modeling, persistence, and reservation locking. The reservation module handles creation, cancellation, expiration, and history queries; the check-in module supports QR-code and location-based verification; the seat module maintains seat queries, status transitions, and status logs; the statistics module provides date-range statistics, area heatmap queries, and daily aggregation; WebSocket events are used for seat status synchronization and reservation notifications; and MQTT is used to ingest device reports from ESP32 nodes equipped with HC-SR501 infrared sensors. On the client side, the WeChat mini-program developed with Taro and React provides seat browsing, reservation, check-in, history viewing, and credit score related entries.

According to the current codebase, the implemented prototype has already completed several key paths, including a 30-minute reservation holding period, automatic expiration release for pending reservations, basic check-in verification, credit score deduction for no-shows, seat status logging, device-side sensor reporting, and real-time seat status synchronization on the mini-program. However, some abilities remain incomplete or only partially prepared, such as automatic release after a long absence, a fully implemented favorites feature, mature user portrait statistics, a complete operational analytics system, fully deployed reminder pushing, and an admin console. Therefore, this thesis describes them conservatively instead of presenting them as completed production functions.

Rather than expanding beyond the real project boundary, this draft is organized strictly around the modules, pages, services, entities, and tests that already exist in the repository. It aims to provide a more formal and structurally complete thesis foundation that can be refined further in subsequent iterations.

Key Words: library; seat reservation; WeChat mini-program; NestJS; MQTT; WebSocket

目录

中文摘要	I
Abstract	II
目录	III
第一章 绪论	1
研究背景	1
研究目的与意义	1
相关系统与现状概述	1
论文主要内容与章节安排	2
第二章 相关技术理论	3
后端框架与模块化组织	3
TypeORM、MySQL 与 Redis 在本项目中的作用	3
MQTT 与 WebSocket 实时通信机制	3
微信小程序、Taro 与前端状态管理	4
ESP32 与 HC-SR501 感知基础	4
第三章 系统设计	6
系统建设目标	6
总体架构设计	6
核心业务流程设计	7
预约与签到主流程	7
感知与同步辅助流程	10
数据对象与实体关系设计	10
状态机与规则约束设计	12
接口与实时事件设计	13
第四章 系统实现	15
后端总体实现	15
微信登录与鉴权实现	15
预约创建、取消与过期处理实现	16
签到校验实现	16
座位状态管理与状态日志实现	17
前端实时同步与状态管理实现	17
MQTT 设备接入与命令控制实现	17
感知判定、信誉分与统计服务实现	18
第五章 系统测试	19
测试环境与测试思路	19
典型测试场景设计	19

预约与签到测试结果分析 20
 状态同步与感知处理测试分析 21
 统计与信誉分相关测试分析 21
 测试结果综合评价 21
 第六章 总结展望 23
 工作总结 23
 当前不足 23
 后续展望 23
 参考文献（初稿整理） 25

第一章 绪论

研究背景

高校图书馆是学生进行课程学习、考试复习与科研准备的重要空间，但实际使用过程中常出现“到馆前不了解余座情况、到馆后找座效率低、离场后座位难以及时回收”等问题。传统管理方式更多依赖现场观察与人工提醒，能够维持基本秩序，却难以满足高峰时段的信息透明和快速流转需求。当图书馆座位资源供给相对固定、学生使用需求持续增长时，单纯依靠线下管理往往难以兼顾公平性与效率。

随着移动互联网和物联网技术逐渐成熟，使用微信小程序承担前端入口、使用后端服务统一处理预约规则、使用低成本感知设备上报座位变化，已经成为智慧学习空间建设中较为可行的组合方案。与单一的网页预约系统相比，此类方案更强调“预约是否真实到场”“座位状态是否及时刷新”“异常行为是否可追溯”等问题，因此具有较强的现实应用价值。

本文课题所依托的仓库已经形成一个围绕图书馆自习座位场景的多端工程：后端使用 NestJS 组织业务模块，小程序负责学生侧交互，ESP32 设备负责现场感知。与只停留在原型展示的方案不同，当前项目已经具备预约创建、超时释放、签到校验、状态广播和信誉分处理等核心代码基础，因此适合作为毕业论文初稿的整理对象。

研究目的与意义

本课题的直接目标，不是凭空设计一个理想化平台，而是基于现有代码整理出一条可信、可说明、可继续完善的系统主线。其一，论文需要从学生使用流程出发，说明系统如何通过微信登录、座位查询和在线预约降低找座成本，从而把“能否快速找到并保留一个可用座位”这一真实问题转化为清晰的业务主线。其二，论文需要从管理规则出发，说明系统如何通过签到校验、预约过期处理和信誉分约束减少无效占座，使座位资源分配不仅有前端入口，也有后端规则支撑。其三，论文需要从工程实现出发，说明系统如何通过 MQTT 与 WebSocket 把现场感知、后端处理和小程序展示串联起来，使系统具备比传统预约平台更强的状态同步能力。其四，论文还需要从毕业设计写作出发，将分散在仓库中的页面、实体、服务和测试整理成结构完整、边界清晰、后续可继续润色的论文初稿。

其意义主要体现在三个层面。应用层面上，该系统关注图书馆高频而具体的座位治理问题，具有明确场景依托。工程层面上，项目综合使用后端服务、缓存、数据库、实时通信、小程序开发与嵌入式联网等技术，能够体现毕业设计的综合训练价值。研究层面上，当前系统虽然并不追求一次性完成全部能力，但通过保守描述已实现链路、明确指出未完成部分，可以为后续正式论文成稿提供更扎实的论证基础。

相关系统与现状概述

从高校公共学习空间管理现状来看，常见系统大致可分为三类。第一类是纯预约系统。这类系统解决了“是否能提前占位”的问题，但通常无法判断用户是否真正到场，也难以感知预约后的离座情况。第二类是依赖人工签到或刷卡签到的系统。这类系统规则较清晰，但过程依赖人工组织或额外基础设施，灵活性较低。第三类是引入传感器感知的智能系统。这类系统在状态感知方面更有优势，但同时面临设备成本、数据抖动、网络稳定性和规则解释边界等挑战。

当前仓库中的实现更接近第三类方案，但其建设策略相对克制：一方面，项目已经打通微信登录、预约、签到、设备上报和实时同步的主链路；另一方面，尚未把收藏、后台管理、成熟统计画像、长时离座自动释放等能力写死为既成事实。对毕业论文初稿而言，这种“先完成核心链路，再谨慎讨论扩展能力”的做法更符合实际，也更便于后续答辩前持续补充。

论文主要内容与章节安排

全文共分为六章。第一章介绍课题背景、研究目的与论文结构；第二章围绕当前系统所使用的核心技术及其在本项目中的具体作用展开；第三章从系统目标、总体架构、业务流程、数据对象、实体关系、状态约束与实时接口等方面进行设计说明；第四章结合仓库中的真实服务、页面和设备程序说明系统实现方式；第五章依据现有测试代码与业务链路，对系统测试情况和结果分析进行归纳；第六章对全文工作进行总结，并对尚未完成的部分和后续改进方向进行展望。

本章小结：本章说明了课题写作的现实背景与边界。本文并不把系统描述成已经大规模落地的成熟产品，而是围绕仓库中已经存在的真实功能组织论文内容，为后续章节展开技术与实现分析打下基础。

第二章 相关技术理论

后端框架与模块化组织

当前项目后端以 NestJS 作为主要开发框架。在 `app.module.ts` 中，可以看到系统已经装配用户、认证、座位、预约、签到、设备、统计和 WebSocket 等模块。这样的模块化组织方式有两个直接作用：一是使业务职责边界更加清晰，便于把预约规则、座位状态、签到校验和设备接入分别建模；二是为后续继续扩展更多能力提供结构基础，避免所有逻辑堆积在单一服务中。

NestJS 的依赖注入机制适合当前项目这种“多个服务之间需要协作但又保持职责分离”的工程结构。例如，预约服务需要调用座位服务更新状态、调用用户服务处理信誉分、调用 WebSocket 网关推送过期事件；签到服务又建立在预约服务和位置服务之上。使用模块化框架能够降低这些依赖之间的耦合度，提高后续维护的可读性。

TypeORM、MySQL 与 Redis 在本项目中的作用

在数据存储层面，MySQL 负责保存用户、座位、预约、状态日志和统计结果等长期业务数据，TypeORM 则负责将这些数据对象组织为实体并提供数据库访问能力。与直接手写 SQL 相比，实体建模更便于在论文中说明数据结构，也方便在服务中表达“预约记录”“座位状态日志”“信誉分明细”等业务概念。

Redis 在本项目中的作用不止一种。首先，预约流程中使用 Redis 锁服务处理并发竞争，避免多个用户同时预约同一座位。其次，感知流程中使用 Redis 列表暂存传感器最近一段时间的观测结果，为“可能离座”和“返回使用”判断提供快速读取能力。也就是说，Redis 在系统中既承担并发控制辅助角色，也承担短时序列缓存角色。

技术组件	在项目中的位置	主要承担的职责
NestJS	后端应用框架	组织业务模块、管理依赖注入、承载 HTTP 与 WebSocket 服务
TypeORM	数据访问层	实体映射、仓储访问、简化预约与座位等核心对象的持久化操作
MySQL	持久化存储层	保存用户、座位、预约、状态日志、统计结果等长期数据
Redis	缓存与并发控制层	预约加锁、感知观测缓存、提升短时状态判断效率

表 1 后端关键技术在本项目中的分工

MQTT 与 WebSocket 实时通信机制

MQTT 是面向物联网场景的轻量级发布订阅协议，特别适合像 ESP32 这样的资源受限设备向服务端上报状态。本项目中，设备端通过 `device/{deviceId}/sensor` 和 `device/{deviceId}/status` 主题发布传感器状态和在线状态；后端订阅这些主题后，根据设备编号定位对应座位，并将消息交给感知处理服务继续分析。与高频 HTTP 轮询相比，MQTT 更适合低负载、异步、持续在线的设备通信场景。

WebSocket 则主要面向用户侧交互。在当前项目中，后端网关定义了 `seat:status`、`reservation:expired` 和 `checkin:reminder` 三类事件；小程序端的 socket 服务会监听这些事件并更新本地状态。这样的设计使得系统不必依赖频繁轮询接口，而是在座位状态变化、预约过期等关键节点主动广播消息，提升了前端展示与后端状态的一致性。

需要注意的是，虽然 `checkin:reminder` 事件已经在后端网关和小程序监听层面形成了接口基础，但这并不等于签到提醒机制已经形成成熟、稳定、全面投入使用的通知系统。对于毕业论文写作而言，应将其表述为“已具备实时事件基础”，而不是“已完成完整提醒产品化”。

微信小程序、Taro 与前端状态管理

本项目将微信小程序作为学生侧主要入口，原因在于其使用门槛低、适合校园场景传播、能够较好承载座位查询、预约和签到等轻量交互。前端采用 Taro 与 React 组合实现页面开发，既保留了 React 组件化组织方式，也能够输出小程序运行产物。对于毕业设计项目而言，这样的技术选型兼顾了开发效率与场景适配性。

在页面层面，项目已经包含首页、座位详情页、签到页、历史记录页、当前预约页、个人中心页和收藏页入口。状态管理方面，小程序通过 store 和 socket 服务维护当前预约、倒计时以及座位实时状态。这样的前端组织方式说明，项目并非单纯把小程序当作静态表单，而是把它作为与后端实时协同的业务终端。

ESP32 与 HC-SR501 感知基础

设备端采用 ESP32 与 HC-SR501 红外传感器构成基础感知节点。ESP32 负责 WiFi 连接、MQTT 连接、命令订阅与状态上报；HC-SR501 负责输出有人或无人变化信号。`main.cpp` 中可以看到，设备程序包含 WiFi 自动重连、MQTT 断线重连、PIR 预热等待、状态心跳上报和命令处理等逻辑，这些细节体现出该设备程序已具备原型阶段所需的基本联网能力。

与将传感器视作“绝对正确”的理想化写法不同，当前项目后端感知服务并未直接以单次采样结果更新座位状态，而是把观测先写入 Redis 列表，再按时间窗与阈值进行判断。这种做法反映了对红外感知抖动、短时误差和人员短暂停顿等现实问题的初步考虑。

本章小结: 本章并非泛泛罗列技术名词, 而是结合项目代码说明各类技术在系统中的实际职责。后续系统设计与实现章节将以这些技术角色为基础展开更具体的业务说明。

第三章 系统设计

系统建设目标

结合当前仓库的业务边界，系统建设目标可以归纳为五个相互衔接的方向。首先，系统需要提供面向学生的座位查询与在线预约能力，降低无效找座时间，使用户在进入图书馆之前即可获知区域余座与单个座位状态。其次，系统需要建立预约保留与超时释放机制，避免预约后长期不签到造成资源浪费，并通过统一后端规则提高资源周转效率。再次，系统需要提供二维码签到和位置签到两种基础校验方式，增强“预约人与到场人一致”的约束，让预约结果真正落到现场使用环节。第四，系统需要建立从设备感知到前端展示的状态同步链路，使座位状态不再完全依赖人工维护，而是能够随着业务状态和现场观测实时更新。最后，系统还需要以信誉分与违规明细为基础，对预约超时等行为形成可追溯约束，为后续治理与统计预留可持续扩展空间。

需要强调的是，建设目标中包含“感知离座行为并为后续规则预留依据”，但这并不代表当前版本已经完整落地“离座超过 1 小时自动释放”。系统已具备相关状态、观测缓存和信誉规则枚举基础，但完整闭环仍待继续实现，因此论文只将其写为设计预留和扩展方向。

总体架构设计

从系统层次上看，当前项目可划分为应用层、平台层和感知层三个部分。应用层负责学生侧交互，平台层负责业务规则、数据存储和实时推送，感知层负责现场状态采集与设备在线维护。三层之间并不是简单的上下堆叠关系，而是围绕“预约—签到—使用—状态回传”构成闭环协同结构。



表 2 系统总体架构图

在该架构中，后端平台层承担了最核心的“规则汇聚”职责。无论用户是通过小程序发起预约，还是设备侧持续上报现场占用变化，真正决定座位状态是否合法、预约是否有效、信誉分是否需要变化的逻辑都集中在服务层处理。这种集中式规则组织方式有助于避免前端、小程序、设备端各自维护一套分散规则，从而降低状态不一致的风险。

从部署逻辑看，MySQL 和 Redis 并非附属组件，而是系统稳定运行的关键基础。一方面，MySQL 负责承载可追溯的长期业务数据，包括预约记录、信誉分明细和状态日志；另一方面，Redis 提供预约并发控制和设备观测缓存，使系统既能处理同一时刻多个用户争抢同一座位，也能对连续传感器观测进行时间窗判断。对于毕业设计而言，这种“双存储角色分工”能够较好体现系统设计的完整性。

核心业务流程设计

预约与签到主流程

用户在小程序首页查看区域列表后，可进入座位详情页选择空闲座位发起预约。后端预约服务先检查座位当前状态是否为可预约状态，再校验用户是否已有待处理预约，之后通过 Redis 锁服务尝试锁定座位。锁定成功后，系统创建预约记录，并把 expiresAt 设置为当前时间后 30 分钟，同时将座位状态变更为 RESERVED。若用户在有效期内完成签到，则预约进入进行中状态；若未签到，则由过期处理逻辑释放座位并执行相应扣分。

签到流程以当前有效预约为前提。小程序端提供二维码签到和位置签到两个入口。后端签到服务会先取得当前预约，再根据签到方式分别完成二维码内容比对或位置距离校验，校验通过后调用预约服务将预约状态改为 ACTIVE，同时将座位状态改为

IN_USE。这种设计使签到既与预约主链路保持一致，又能通过不同校验方式适配图书馆现场部署条件。

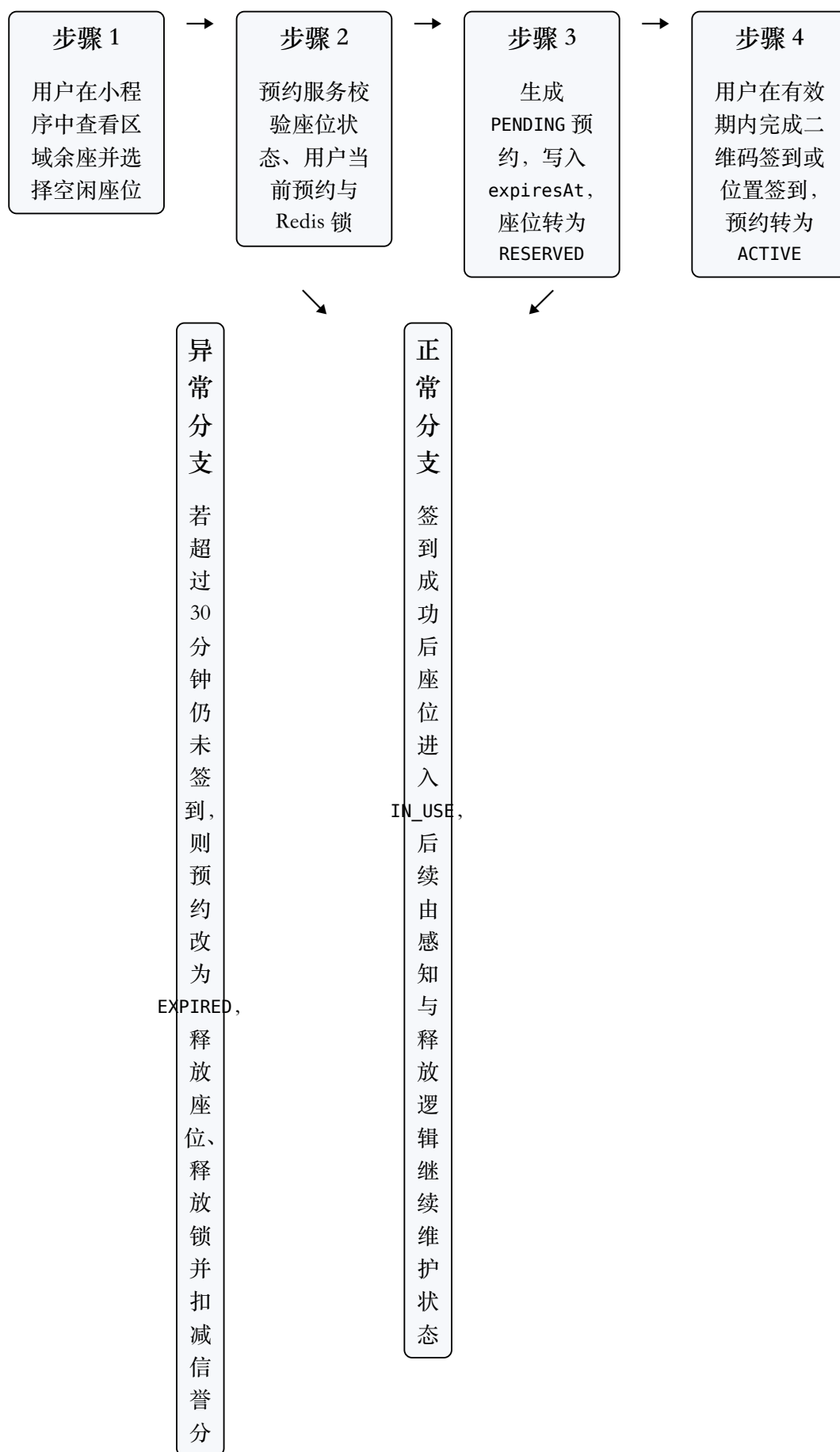


表 3 预约—签到—过期处理业务流程图

从业务约束角度看，预约流程并非单纯的“先到先得”接口调用，而是把并发锁、预约有效时长、用户唯一待处理预约限制和签到校验统一串联起来。这样做的意义在于，系统可以从预约发起时就开始控制资源占用边界，而不是等到用户未到场之后再被动清理。这也是当前仓库中最完整、最适合在论文中作为核心主链路展开的部分。

感知与同步辅助流程

设备端在 WiFi 和 MQTT 连接建立后，会按占用变化上报传感器状态，并按固定间隔发布在线状态心跳。后端感知服务收到消息后，先根据设备编号找到对应座位，再把观测写入 Redis 列表。对于正在使用中的座位，如果一段时间内连续观测到“无人”比例较高，则座位状态可能被调整为 MAYBE_LEAVE；如果后续又持续观测到“有人”，则可恢复为 IN_USE。每次状态变化后，座位服务都会写入状态日志并通过 WebSocket 广播到小程序端。

该辅助流程的价值在于，它把物理世界中的“人是否还在座位附近”引入了系统判断，但又没有简单地以单次红外结果替代业务规则。换言之，设备观测在当前系统中主要承担“提供辅助证据”的角色，而最终是否产生状态迁移，仍由后端服务根据时间窗、阈值和当前业务状态统一处理。这样的设计既体现了工程克制，也为后续扩展长时间离座治理预留了较清晰的接口位置。

数据对象与实体关系设计

当前项目中与论文主线最相关的数据对象包括用户、座位、预约、座位状态日志和信誉分明细。为避免泛化描述，本文重点说明与业务闭环直接相关的字段语义，并进一步结合实体间关系说明数据如何支撑预约、签到、扣分与统计。

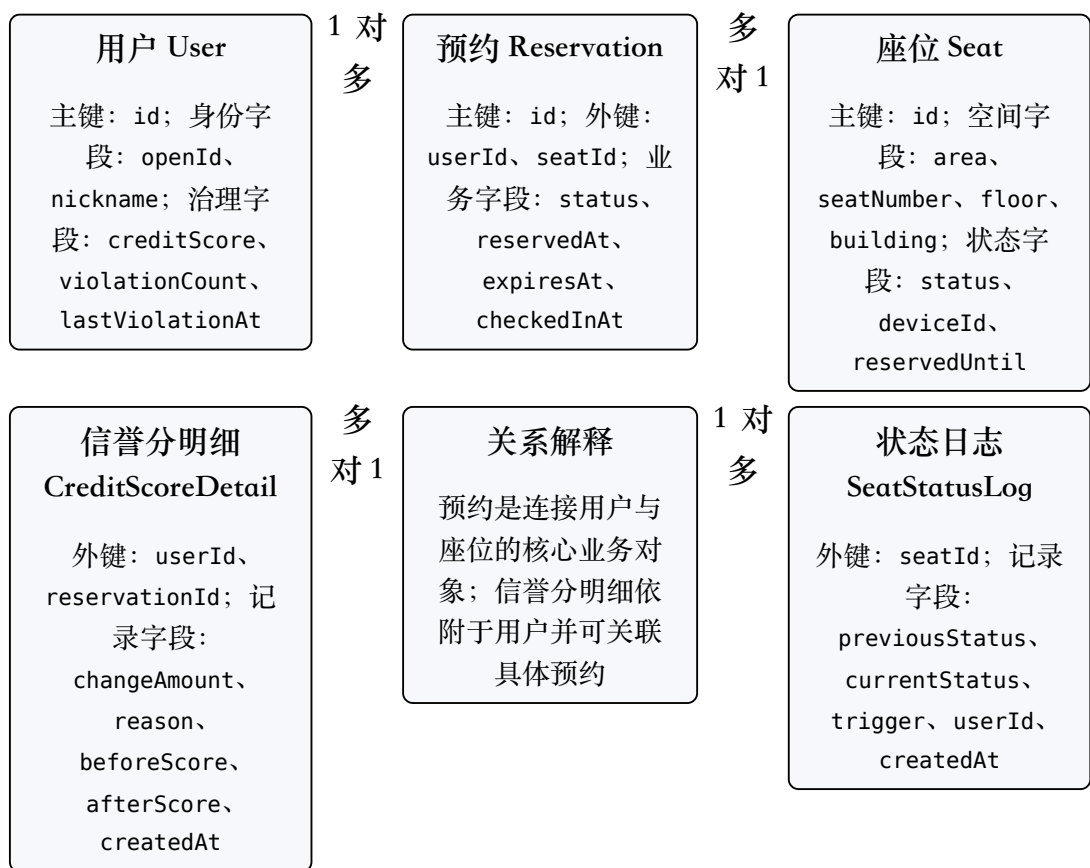


表 4 核心实体 ER 图

从实体关系可以看出，预约记录是系统业务建模的中心。它既连接用户与座位，也承载预约是否有效、是否已签到、是否已过期等状态信息。签到服务并没有单独维护一张新的复杂业务表，而是以预约状态迁移和时间字段更新为主，这使得主流程结构相对清晰，也便于在论文中对业务生命周期进行说明。

座位实体不仅负责承载基础空间属性，还保存设备绑定关系、当前使用用户和预约截止时间等运行态字段。这说明座位表并不是静态资源清单，而是系统状态切换的重要承载体。状态日志表则为这种“运行态更新”提供可追溯记录，为后续统计服务、异常定位与论文中的测试分析提供依据。

对象	关键字段	设计意图
用户	id、nickname、creditScore、violationCount	记录基础身份与信誉分情况,为预约资格与违规处理提供依据
座位	area、seatNumber、status、deviceId、reservedUntil、latitude、longitude	描述座位位置、设备绑定、当前状态和签到校验基础信息
预约	userId、seatId、status、reservedAt、expiresAt、checkedInAt	承载预约生命周期,是预约、签到与历史记录的核心对象
状态日志	previousStatus、currentStatus、trigger、userId	记录每次座位状态变化,为统计和追踪问题提供依据
信誉分明 细	changeAmount、reason、beforeScore、afterScore、reservationId	实现扣分过程可追溯,而不是只保留最终结果

表 5 核心数据对象设计

状态机与规则约束设计

在状态设计上,系统至少涉及 FREE、RESERVED、IN_USE、TEMP_LEAVE 和 MAYBE_LEAVE 等状态。这里需要区分两个层面:一类是直接体现在预约与签到主链路中的状态,如空闲、已预约、使用中;另一类是主要服务于感知判断和后续规则扩展的状态,如可能离座、暂离。论文写作中必须把二者区分开来,不能把“存在状态枚举”直接等同于“完整治理规则已经落地”。

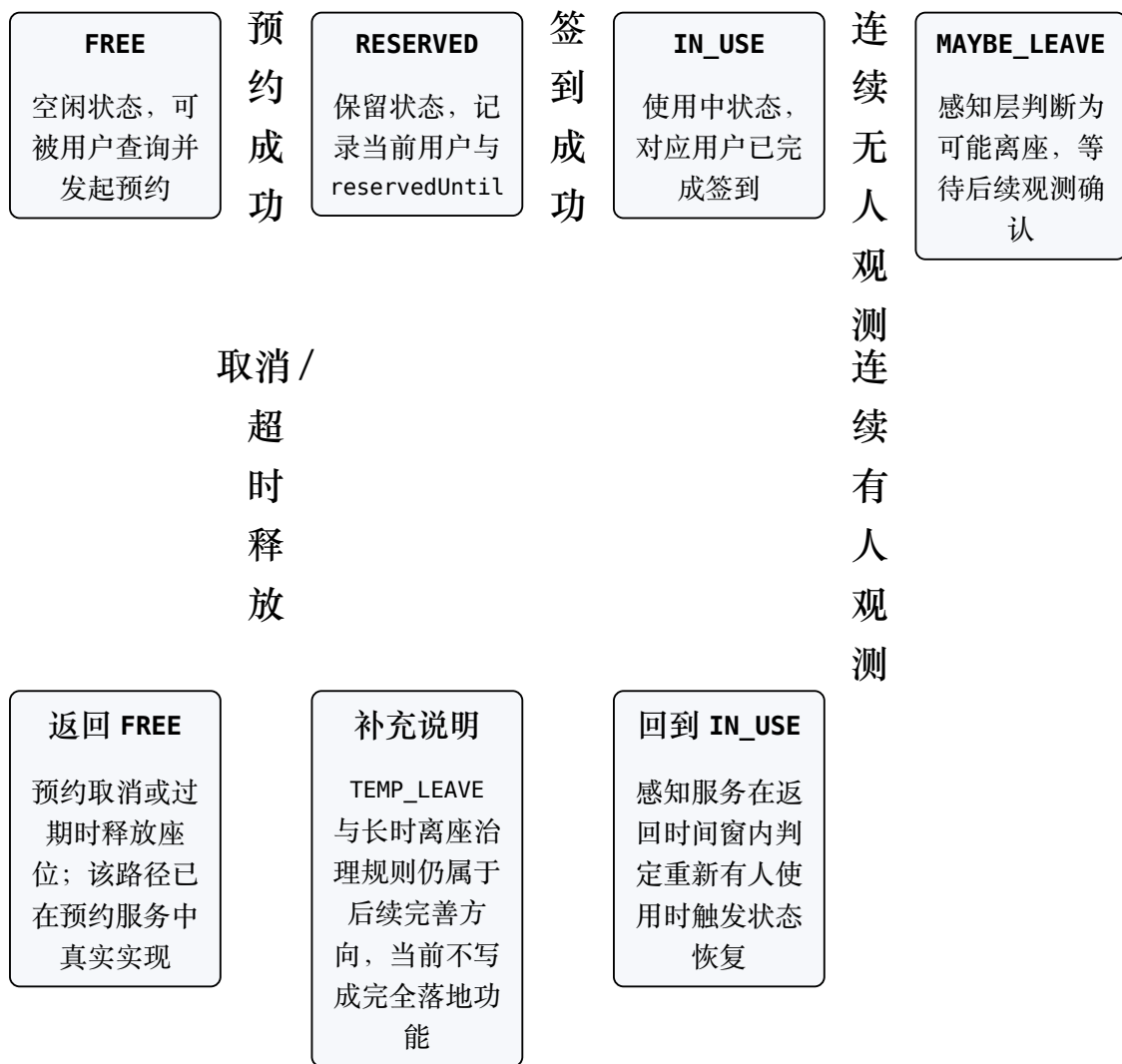


表 6 座位状态迁移图

除了状态枚举本身，系统还通过规则约束限制非法状态切换。例如，预约服务在创建预约前会检查座位当前是否处于 FREE 状态，并检查用户是否已经存在待处理预约；签到服务则要求当前预约存在且未过期，才能将预约从 PENDING 转为 ACTIVE。也就是说，系统中的状态迁移不是任意可达的，而是与角色、时间窗口、当前状态和业务上下文共同绑定。

从论文写作角度看，状态机设计的价值在于将“页面按钮操作”转化为“状态受约束的业务对象流转”。这样不仅可以更正式地描述系统工作原理，也能为第五章测试分析提供更自然的论证结构，因为测试本质上就是在验证状态迁移条件是否成立、异常分支是否被拦截以及日志是否被正确记录。

接口与实时事件设计

系统不仅依赖普通接口，也依赖实时事件维持状态同步。围绕当前仓库实现，较关键的交互可整理如下表。

接口或事件	所属层	作用说明
/user/login	后端业务接口	接收微信登录 code, 调用微信会话接口获取 openid, 再签发 JWT 令牌供小程序后续访问
/reservations 相关接口	后端业务接口	创建预约、取消预约、查询当前预约与历史记录, 是主流程入口
/reservations/{id}/checkin	后端业务接口	处理二维码签到与位置签到请求, 承接预约到使用中的状态变更
seat:status	WebSocket 实时事件	广播座位状态变化, 小程序据此刷新座位可用状态
reservation:expired	WebSocket 实时事件	通知用户预约过期并刷新当前预约数据
checkin:reminder	WebSocket 实时事件	为签到倒计时同步提供事件基础, 当前仍属于基础能力而非成熟提醒系统
device/{id}/sensor	MQTT 主题	设备上传传感器状态, 驱动后端感知逻辑
device/{id}/status	MQTT 主题	设备上传在线状态与心跳, 便于判断节点连接情况

表 7 系统关键接口与实时事件

对该设计进行归纳, 可以发现系统交互被分成“请求—响应型”与“事件推送型”两条链路。前者解决登录、查询、创建预约和签到校验等需要立即得到处理结果的操作; 后者解决状态广播、预约过期通知和倒计时同步等更适合由服务器主动下发的变化信息。正是这两条链路的组合, 才使系统既保留了传统业务接口的确定性, 也具备了较好的实时感知能力。

本章小结: 本章围绕当前系统的真实目标和结构, 给出了论文后续论证所依赖的设计基础。重点不在于追求功能数量, 而在于把主链路、实体关系、状态边界、实时接口与扩展预留说明清楚。

第四章 系统实现

后端总体实现

从工程结构上看，后端入口模块统一引入了配置模块、数据库连接、Redis 提供者以及用户、认证、座位、预约、签到、设备、统计和 WebSocket 模块。这表明系统已经具备较明确的服务划分。论文写作时，可以把后端实现概括为三类协作：一类是面向用户业务的预约、签到和座位管理；一类是面向设备接入的 MQTT 与传感处理；另一类是面向同步和统计的 WebSocket 事件与统计服务。

这种实现方式的优势在于，主流程和辅助流程并未混杂在一起。例如，预约过期时由预约服务负责状态处理，但状态广播由 WebSocket 网关负责；设备感知由传感处理服务判断，但真正修改座位状态仍通过座位服务完成。这样的职责划分有利于论文中分别分析业务链路和技术链路。

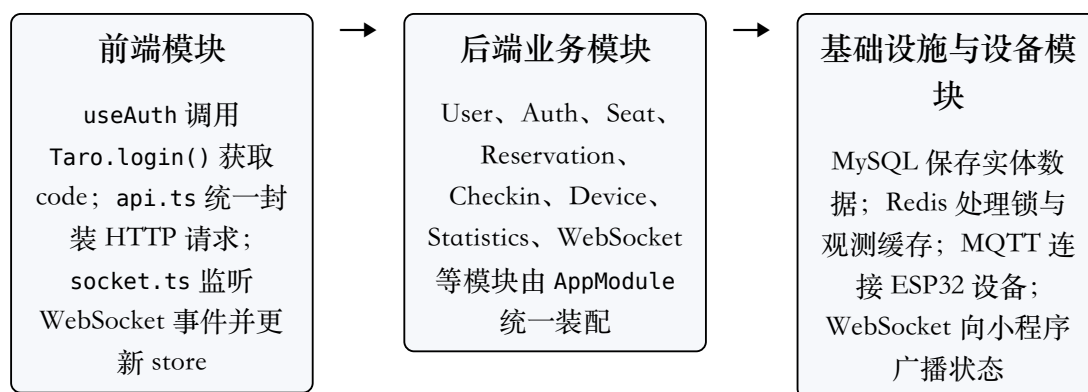


表 8 前端—后端—设备模块关系图

该模块关系图强调的不是文件目录层面的隶属，而是运行时依赖链。前端侧既依赖 HTTP 接口完成显式业务操作，也依赖 WebSocket 事件感知后端状态变化；后端侧既要处理用户请求，又要持续消费设备消息；而数据库与缓存则为这些模块提供统一的状态持久化和并发控制基础。这样的实现结构说明，项目虽然规模有限，但已经具备一个完整信息系统应有的基本协作形态。

微信登录与鉴权实现

在登录链路上，小程序端 `useAuth` 通过 `Taro.login()` 获取微信临时登录凭证 code，然后调用 `userStore` 中的 `login` 方法，再由 `api.ts` 发送 `/user/login` 请求。后端 `user.controller.ts` 收到 code 后，会调用 `WechatService.code2Session` 请求微信接口换取 `openid` 与 `session_key`，随后通过 `UserService.createOrUpdate` 创建或更新本地用户，并用 `JwtService` 生成访问令牌。这一链路表明，系统当前的身份识别并不是本地虚拟账号，而是真正围绕微信登录凭证建立用户映射。

登录完成后，前端将访问令牌写入本地存储，并在后续请求中通过 `Authorization: Bearer <token>` 方式附带到接口调用。后端 `JwtAuthGuard` 负责统一解析并校验令牌，将用户身份注入请求上下文，以便 `/user/profile`、`/user/credit` 和预约相关接口使用。对论文写作而言，这一实现足以支撑“已实现微信登录与 JWT 鉴权主链路”的表述。

需要克制说明的是，当前仓库中能够明确验证的是登录、令牌签发与令牌校验基础链路，而不是一个已经覆盖复杂风险控制、设备指纹识别和多端会话管理的成熟认证中心。因此，论文只把这部分写成“适合原型阶段的轻量身份鉴权实现”。

预约创建、取消与过期处理实现

在 `reservation.service.ts` 中，预约创建流程具有较清晰的顺序：先读取座位并判断是否空闲，再判断用户是否已有待处理预约，然后调用 Redis 锁服务尝试锁定座位。锁定成功后，系统以当前时间加 30 分钟生成过期时间，创建状态为 `PENDING` 的预约记录，并调用座位服务将座位状态更新为 `RESERVED`。这部分实现说明“预约保留 30 分钟”并不是文档设想，而是已经真实写入服务逻辑的业务规则。

取消预约时，系统要求操作人必须是预约本人，且预约状态仍允许取消。取消完成后，后端会释放 Redis 锁并通过座位服务将座位重新置为空闲状态。对于过期处理，服务会扫描当前已超时而仍为待签到状态的预约，将其状态改为 `EXPIRED`，释放座位、释放预约锁、扣减信誉分，并发出预约过期事件。这一实现形成了“预约创建—等待签到—超时释放”的完整基础闭环。

从实现细节看，该服务真正体现了业务规则集中化的设计思想。并发约束、时间窗口、状态回收、信誉惩罚和事件通知都落在一个连续可追踪的处理序列中，而没有分散到多个互不关联的前端页面或脚本逻辑中。这使论文在描述系统实现时，不再只是罗列接口，而是能够围绕一条清晰的后端业务链展开分析。

签到校验实现

签到服务建立在当前预约存在的前提上。代码表明，服务首先通过预约服务获取当前预约并核对预约编号，然后根据请求中声明的签到方式进行不同校验。若采用二维码签到，则系统拼接 `SEAT_座位 ID_区域_座位号` 作为预期内容进行比对；若采用位置签到，则使用座位的经纬度与用户上报位置进行距离校验。校验通过后，服务再调用预约服务执行真正的签到状态流转。

这种“签到逻辑分层”方式具有较好的工程可维护性。二维码与位置校验只负责判定用户是否满足签到条件，而预约状态变更仍交由预约服务统一完成，因此不会出现签到服务和预约服务分别维护两套状态切换逻辑的问题。同时，二维码签到和位置签到共用同一条预约主线，也为后续继续扩展更多现场校验方式保留了接口位置。

这里需要特别强调两个写作边界。其一，项目已经完成了二维码校验和位置校验的基础逻辑，因此可以写为“已实现基础签到校验”；其二，签到在真实馆内环境中的稳定

性还会受到二维码布设、定位误差、移动网络和用户终端权限等影响，因此不应在论文中写成“签到能力已完全成熟并大规模投入运行”。

座位状态管理与状态日志实现

座位服务是整个项目状态一致性的关键环节。无论是预约成功、签到成功、预约取消、预约过期，还是设备感知触发的状态变化，最终都通过座位服务修改座位当前状态。服务在更新状态时，不仅会写回座位表，还会同步保存一条状态日志，记录前一状态、当前状态、触发原因和关联用户。这种实现方式使状态变化具备可追溯性，也是后续统计服务能够工作的重要前提。

此外，座位服务提供按区域筛选座位、按状态筛选座位和汇总区域可用数量的能力。对小程序首页而言，这些接口是区域列表和座位矩阵展示的主要数据来源；对论文写作而言，这说明系统前端展示并非静态样例，而是确实依赖后端状态聚合结果。也正因为状态聚合是通过服务实时计算得到的，系统才能在预约创建、签到成功或状态释放后快速将结果反馈到前端。

如果把这一实现放在更完整的系统视角下理解，座位服务实际上承担了“统一状态写入口”的职责。无论外部触发源来自用户操作还是设备观测，只要最终需要改变座位当前状态，就必须经过该服务。这样做有助于保证日志记录、广播推送和后续统计依赖的数据格式保持一致，避免不同模块直接修改数据库而产生不可追溯的状态漂移。

前端实时同步与状态管理实现

小程序端已经形成较完整的用户侧页面骨架。首页负责展示区域与座位列表，座位详情页承接预约动作，签到页处理二维码或位置签到，历史记录页用于回看历史预约，个人中心页展示用户头像、信誉分与预约相关入口，收藏页则保留了后续扩展位置。这样的页面结构与后端服务分工基本一一对应，说明前端并非孤立原型，而是围绕真实接口组织而成。

更值得关注的是实时同步实现。socket.ts 中已经监听 seat:status、reservation:expired 和 checkin:reminder 三类事件，其中 seat:status 用于更新局部座位状态，reservation:expired 用于清理当前预约并刷新历史记录，checkin:reminder 用于维护前端倒计时。与此同时，useSeatStore 与 useReservationStore 负责缓存当前座位与预约数据，使页面不必在每次交互后都完全重新请求所有信息。

这种“接口请求获取初始数据、Socket 事件维护增量变化”的实现方式，是当前系统在用户体验层面的重要特点。它使小程序首页能够较快反映后端状态变化，也让预约过期这类时效性较强的业务结果更容易及时反馈到用户侧。需要保守表述的地方主要有两点：一是个人中心中的月度使用、总时长和常去区域目前仍以占位显示为主；二是收藏页面尚未形成功能闭环，因此都不应被写成已有成熟统计或偏好功能。

MQTT 设备接入与命令控制实现

在设备接入链路中，ESP32 端程序负责维护 WiFi 连接、MQTT 连接、状态心跳和红外采样。程序对外发布的主题包括设备在线状态和传感器状态，同时还可以订阅服务器下发的命令主题，用于获取状态、同步时间和重启等基础控制。虽然当前实现仍处于原型接入阶段，但已经覆盖了一个联网设备在校园现场运行时最基本的连接维护问题。

具体来看，main.cpp 中对 WiFi 连接和 MQTT 连接都设置了周期性重连逻辑，并在设备启动后等待 PIR 预热完成再开始正式采样。设备程序还会定时上报在线状态心跳，保证服务器能够区分“设备仍在线但无占用变化”和“设备已经离线无法上报”这两类不同情况。对于论文而言，这些实现细节能够明显增强设备章节的可信度，因为它们体现出项目并非只做了一个会发一次消息的演示脚本，而是考虑了现场设备长期运行所需的基础稳态机制。

命令控制方面，设备端支持 get_status、sync_time 和 reset 等基础命令。尽管这还不足以构成完整的设备管理平台，但已经说明系统在架构上预留了“后端可对设备发起控制”的通道。对于毕业设计原型，这种双向通信能力具有较强展示价值，也为后续补充巡检、批量管理或故障恢复策略提供了扩展空间。

感知判定、信誉分与统计服务实现

后端 sensor-processor.service.ts 体现了感知逻辑的关键思路。服务会把最近一段时间的观测写入 Redis 列表，并按照不同的时间窗和阈值判断是否“可能离开”或“已经返回”：离开判断使用 5 分钟窗口和 0.8 阈值，返回判断使用 3 分钟窗口和 0.7 阈值。这说明项目已经在工程上考虑到连续观测和抗抖动，而不是仅依据单次传感器读数直接决定座位状态。

用户服务中已经定义了预约未到场、签到入座不符、长时间离座和远程签到等多类违规类型，并为每类违规行为给出扣分值。现阶段真正接入主链路并已明确可验证的，是预约超时未签到带来的信誉分扣减：当预约过期处理触发时，系统会更新用户信誉分、累计违规次数，并写入信誉分明细表。这样一来，信誉约束既体现在用户分值变化上，也体现在明细记录可追溯上。

统计服务部分已经实现按日期区间查询座位统计、按区域查询热力数据和按日生成统计结果等基础功能，并且生成逻辑依赖座位状态日志。这说明项目已经具备统计服务框架，但从当前实现深度来看，仍更适合表述为“基础统计能力”或“统计接口雏形”，而不应将其描述为成熟运营分析系统。同样，感知服务虽然已经能把“连续无人”与“连续有人”转化为中间状态判断，但并不意味着基于长时间离座的完整自动释放策略已经投入运行。

本章小结：本章从后端、小程序和设备三个方向说明了系统当前已经落地的实现基础。真实可写的主线包括微信登录、预约、签到、过期处理、设备上报、状态同步和信誉分约束；需要克制的部分则包括完整收藏、成熟统计画像、后台管理和长时离座自动释放等扩展能力。

第五章 系统测试

测试环境与测试思路

当前项目尚处于毕业设计原型完善阶段，因此测试重点放在业务正确性与主链路一致性，而不是性能压测或大规模部署验证。结合仓库中的测试文件，可以确认系统已围绕预约服务、签到服务、座位服务、传感处理服务、统计服务和用户服务编写单元测试。这些测试能够覆盖系统最关键的规则判断与状态变化逻辑，适合作为论文中“已有测试基础”的主要依据。

在论文层面，测试思路主要遵循以下原则：第一，优先验证核心业务路径是否成立，例如预约是否成功、过期是否释放、签到是否校验通过；第二，验证关键异常分支是否得到处理，例如二维码错误、位置过远、无当前预约等情况；第三，验证实时链路和感知链路是否具备基本可解释性，例如状态事件是否被广播、感知判断是否依赖观测序列而非单次读数。换言之，本章更重视“规则是否正确实现”，而不是“系统是否已经在真实校园场景中完成大规模运行实验”。

测试对象	仓库中的对应测试	主要关注内容
预约服务	reservation.service.spec.ts	预约创建、取消、签到成功、过期处理、历史查询
签到服务	checkin.service.spec.ts	二维码签到、位置签到、无预约、二维码错误、位置过远
座位服务	seat.service.spec.ts	状态更新、区域汇总、日志记录、释放和预约状态处理
感知处理	sensor-processor.service.spec.ts	观测写入、离开判定、返回判定、未绑定设备处理
统计服务	statistics.service.spec.ts	区间统计、热力查询、按日生成统计结果
用户服务	user.service.spec.ts	用户创建更新、信誉分扣减、信誉分明细查询

表 9 当前仓库中的主要测试覆盖范围

典型测试场景设计

为了让测试分析更接近论文表达，而不是仅停留在“有无测试文件”的说明层面，本文进一步将主链路抽象为几类典型测试场景。第一类是预约成功场景，即座位为空闲、用户没有其他待处理预约且加锁成功时，系统是否能够正确写入预约记录并更新座位状态。第二类是预约失败场景，即座位已被占用、用户已有预约或并发加锁失败时，系统是否能够正确拦截并返回明确错误。第三类是签到场景，即用户在预约有效期内通过二

维码或位置校验后，系统是否能够将预约转为有效使用状态。第四类是过期释放场景，即超过预约保留时间后，系统是否能够同步完成状态回收、锁释放与信誉分处理。第五类是感知辅助场景，即设备持续上报的观测序列是否会影响座位中间状态，并是否遵循时间窗阈值而非单次读数。

场景编号	输入条件	预期行为	论文中的分析重点
S1	空闲座位 + 合法用户 + 锁获取成功	创建预约、写入过期时间、座位转为 RESERVED	验证主流程的起点是否成立
S2	同一用户重复预约或目标座位已不可用	拒绝创建预约并返回错误	验证并发与资源约束是否有效
S3	预约有效期内执行二维码或位置签到	预约转为 ACTIVE、座位转为 IN_USE	验证签到是否与预约主线正确衔接
S4	超过 30 分钟未签到	预约转为 EXPIRED、座位释放、信誉分扣减	验证资源回收与惩罚链路是否闭环
S5	连续无人 / 连续有人观测序列	进入 MAYBE_LEAVE 或恢复 IN_USE	验证感知逻辑是否基于时间窗判断

表 10 典型测试场景设计表

通过这种场景化整理，可以看到测试并不是彼此孤立的断言集合，而是对应于论文第三章中已经抽象出的预约状态机、签到规则和感知辅助流程。测试章节因而能够与设计章节形成一一映照关系，从而增强整篇论文的结构完整性。

预约与签到测试结果分析

从测试文件可见，预约服务已经覆盖预约成功、座位不可预约、用户已有预约、加锁失败、取消预约、预约过期和历史记录分页等典型场景。其中较关键的是过期处理：测试不仅验证预约状态会变更为过期，还验证座位会被释放、预约锁会被释放、信誉分会被扣减并触发预约过期事件。由此可以说明，超时释放并不是单点修改，而是涉及数据、锁、用户约束和实时通知的协同结果。

签到服务测试覆盖二维码签到成功、位置签到成功、当前预约不存在、预约编号不匹配、二维码无效和位置过远等情况。这说明当前签到功能已经形成基础校验闭环。论文在表述测试结果时，应说明“基础校验逻辑已具备测试支撑”，而不应继续上升为“现场签到可靠性已被完全验证”。

进一步看，预约与签到测试共同证明了系统最重要的一条状态迁移链：用户在空闲座位上创建预约，预约在有效时间内允许签到，签到成功后进入使用中，超过时限则进入过期释放。这条链路之所以重要，是因为它直接体现了图书馆座位治理最核心的公平性诉求，即预约不应无限期占有资源，签到必须与真实到场行为建立绑定关系。

状态同步与感知处理测试分析

感知处理测试验证了设备绑定座位和未绑定座位两类分支，并对 Redis 观测写入、离开判定和返回判定进行了单元验证。特别是离开判断和返回判断，测试通过构造不同占用比例的观测样本来校验阈值逻辑，这与服务中的时间窗判断思路相吻合。由此可见，系统确实在代码层面对“短时抖动不能直接触发状态切换”做了初步控制。

实时同步方面，虽然当前仓库并未直接提供完整的端到端联调报告，但从后端 WebSocket 网关定义事件和小程序 socket 服务监听事件这两个实现点可以判断：系统已经具备“服务端广播—客户端更新本地状态”的基础同步机制。对于论文写作，这足以支撑“已实现基础实时同步”，但不足以支撑“已形成高并发实时通知平台”等更强表述。

从测试与实现的对应关系看，感知链路和实时同步链路都更接近“支持主链路稳定运行的辅助能力”。它们不直接替代预约和签到规则，但能提升系统对现场变化的响应速度与解释能力。因此，在论文评价体系中，这部分更适合被写成“增强系统真实性与完整性的工程能力”，而不是“已经充分验证的核心治理闭环”。

统计与信誉分相关测试分析

统计服务测试涵盖按日期区间获取座位统计、按区域筛选、按座位筛选、获取热力数据以及生成日统计等场景，说明系统已具备基础统计服务能力。与此同时，代码也表明统计生成目前主要依赖状态日志进行相对基础的聚合，因此论文中更适合将其表述为“为后续报表和分析提供数据基础”。

信誉分相关逻辑主要通过用户服务和预约过期处理协同验证。当前可以较为明确地写入论文的结论是：系统已经将预约超时未签到与信誉分扣减打通，并且具备信誉分明细查询入口。至于更复杂的多规则综合信誉治理，还需要在后续开发中继续补充。

如果从论文论证角度总结，这一部分测试的意义不在于证明系统已经拥有复杂成熟的运营分析能力，而在于证明项目已经把治理所需的基础数据链条建立起来：用户侧有信誉分变化记录，座位侧有状态日志，统计侧有基础聚合服务。也正因为这些底层数据结构已经存在，后续扩展才具有现实可行性。

测试结果综合评价

结合现有测试与代码结构，可以得到以下较为稳妥的分析结论。第一，系统的预约、签到、过期释放与信誉分约束已经形成明确主链路，具备继续联调和完善的基础。第二，感知处理已经从“单次传感器值”上升到“短时序列判断”，说明项目对误判问题已有初步工程化考虑。第三，实时同步具备明确事件和前端监听基础，但其完整使用效果仍依赖

实际部署和联调验证。第四，统计和个人画像相关内容已有基础数据接口或页面入口，但现阶段不宜写成成熟运营分析结果。

因此，第五章的结论应是“当前系统已经具备基础测试支撑和主链路可验证性”，而不是“所有功能均已完成充分实验验证”。这种结论更加符合仓库现状，也更有利于后续完善论文内容。对于一篇毕业论文初稿而言，如实区分“已有测试支撑的实现”与“尚待联调验证的扩展能力”，比单纯追求表面完备更为重要。

本章小结：本章依据仓库中的测试文件和实现链路，对系统已有验证基础进行了整理。测试能证明主流程具备正确性基础，但不能替代大规模部署、性能评估和长期运行实验，这一点需要在论文中如实说明。

第六章 总结展望

工作总结

本文围绕图书馆自习座位智能预约系统，对当前仓库中的真实实现进行了结构化整理，并重写形成一篇更完整的毕业论文初稿。从系统现状来看，项目已经形成以下几条较清晰的主线：一是以微信登录、小程序座位查询和预约发起构成用户入口；二是以后端预约服务、签到服务和座位服务构成主要业务规则；三是以 Redis 锁和 Redis 观测缓存分别支撑预约并发控制和感知判断；四是以 WebSocket 实现前端状态同步；五是以 ESP32 与 HC-SR501 实现基础现场感知接入；六是以信誉分明细和统计服务为后续治理和分析预留延展空间。

与原有初稿相比，本文在写作上更强调前后置结构完整性、章节组织清晰度、图表支撑与设计解释力度，同时避免使用与当前仓库无关的业务内容，也避免把尚未完成的能力写成既成事实。换言之，本次重写的重点不只是“字数变多”，而是让论文初稿更接近真实工程、真实边界与真实可继续完善的毕业设计文本。

当前不足

尽管项目已经具备较完整的主链路，但仍存在若干不足，需要在后续工作中继续说明和改进。首先，长时间离座自动释放尚未形成完整闭环。当前系统已有感知缓存、状态判断与信誉规则枚举基础，但尚未把“一小时未回座自动释放”真正固化为完整业务链路。其次，收藏功能尚未完成。收藏页仍处于保留入口和“功能开发中”状态，因此不能作为已完成功能写入成果描述。再次，个人中心统计信息仍较基础，月度使用、总时长、常去区域等卡片目前主要体现页面预留，不代表已有真实画像结果。与此同时，统计服务仍以基础聚合为主，它为后续分析提供了接口和数据基础，但离成熟运营分析系统仍有明显距离。除此之外，签到提醒与实时推送虽然已有接口基础，但完整触达机制、提醒策略和稳定效果还需要继续验证；后台管理端也尚未纳入本次可验证实现范围，不应在论文中写成已经完成的子系统。

这些不足并不意味着项目缺乏价值，而是说明当前系统更适合被定位为“主链路较清晰、扩展链路待继续补足的工程原型”。毕业论文初稿若能够如实呈现这种状态，反而更利于后续按真实进度继续完善，而不会因为前期表述过满导致后续答辩或联调时产生明显偏差。

后续展望

围绕上述不足，本文认为后续工作可从以下几个方向继续推进。第一，补齐长时间离座处理链路。在已有观测与状态基础上，把“暂离—长期未回—自动释放—信誉处理”真正串联起来，并结合现场管理规则优化阈值。第二，完善用户侧能力，逐步将收藏、个人画像和更多历史分析从页面占位发展为可用功能。第三，增强统计维度，基于预约

记录和状态日志构建更完整的区域热力、利用率变化和违规分布分析。第四，补充馆员或后台管理端，为异常处理、设备巡检、规则配置和人工干预提供操作入口。第五，开展更多真实场景联调，验证定位签到、设备联网稳定性和实时同步体验。

总体而言，当前项目已经超出了单纯“界面展示型毕设”的范畴，具备从设备上报到后端处理再到前端同步的完整基础框架。只要后续继续围绕真实需求、真实边界与真实验证推进，该系统具备进一步发展为校园图书馆学习空间管理子系统的潜力。

本章小结：本章对论文和项目现状进行了收束。对于已经实现的部分，本文给出基于代码的稳妥总结；对于尚未实现或仅完成预留的部分，本文明确指出边界，为后续继续完善留下空间。

参考文献（初稿整理）

以下条目为当前论文继续成稿时拟重点整理和规范化补录的参考资料方向，现阶段先保留已知来源，以避免凭空虚构文献内容。

- [1] NestJS Documentation. <https://docs.nestjs.com/>
- [2] TypeORM Documentation. <https://typeorm.io/>
- [3] MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/>
- [4] Redis Documentation. <https://redis.io/docs/>
- [5] MQTT Version 3.1.1 / 5.0 Specification. <https://mqtt.org/>
- [6] Taro 官方文档. <https://docs.taro.zone/>
- [7] ESP32 Programming Guide. <https://docs.espressif.com/>